

BÁO CÁO BÀI TẬP LỚN SIMPLE OPERATING SYSTEM

Trần Nhật Bảo Anh – MSSV 2410155

Tháng 6 năm 2026

Contents

1 Tổng quan

Dự án mô phỏng một hệ điều hành đa nhiệm gồm MLQ scheduler, unified system call, phân tách user/kernel space và paging 64-bit năm cấp. Mỗi CPU được mô phỏng bởi một thread. Scheduler queue, danh sách process đang chạy và physical frame free-list được bảo vệ bằng mutex.

Mỗi process sở hữu một `proc->mm`. Kernel không thay đổi một con trỏ `krnl->mm` dùng chung khi context switch, do đó nhiều CPU không thể vô tình thao tác trên address space của process khác.

2 Trả lời câu hỏi lý thuyết

2.1 Unified system-call interface

Ưu điểm là cung cấp một giao diện thống nhất, giảm coupling giữa chương trình user và implementation trong kernel, đồng thời tạo một điểm kiểm tra quyền truy cập tập trung. Nhược điểm là phát sinh overhead do trap, dispatch và validation; giao diện chung cũng có thể không biểu diễn tốt các đặc tính riêng của từng thiết bị.

2.2 System call thực thi quá lâu

Hệ điều hành dùng timer interrupt hoặc watchdog để phát hiện. Kernel có thể preempt tác vụ nếu kiến trúc cho phép, chuyển process sang trạng thái blocked khi chờ I/O, hoặc kết thúc process nếu vi phạm giới hạn. Kernel code phải tránh giữ lock trong thời gian dài để không chặn toàn hệ thống.

2.3 Lợi ích của các chính sách MLQ

Process có giá trị priority nhỏ được phục vụ trước. Số slot của mức i bằng $\text{MAX_PRIO} - i$, nên mức ưu tiên cao nhận nhiều CPU time hơn. FIFO trong từng queue tạo fairness giữa các process cùng mức. Khi các queue đang có process đều hết slot, scheduler reset slot để process priority thấp vẫn có cơ hội chạy.

2.4 Paging và continuous allocation

Continuous allocation dịch địa chỉ đơn giản nhưng gây external fragmentation và khó cấp phát vùng lớn. Paging loại bỏ external fragmentation và hỗ trợ swapping, đổi lại có internal fragmentation, chi phí page table và nhiều lần memory access khi dịch địa chỉ.

2.5 Kết hợp segmentation và paging

Segmentation phản ánh cấu trúc logic như code, data, heap và stack, thuận lợi cho protection. Paging quản lý physical memory theo frame cố định. Kết hợp hai cơ chế giữ được protection theo segment nhưng tránh external fragmentation khi nạp segment vào RAM.

2.6 Lợi ích của N-level paging

Không gian địa chỉ 64-bit rất thưa. Hierarchical paging chỉ cần tạo các nhánh được sử dụng thay vì một flat page table khổng lồ. Đánh đổi là mỗi translation phải truy cập nhiều bảng; TLB có thể giảm chi phí này.

2.7 Hậu quả khi thiếu synchronization

Hai CPU có thể lấy cùng PCB khỏi ready queue hoặc lấy cùng physical frame khỏi free-list. Kết quả là process chạy đồng thời trên hai CPU, page table bị ghi đè, double-free hoặc data corruption. Implementation dùng `queue_lock` cho scheduler và mutex cho frame allocator cùng các thao tác cập phát memory.

3 CPU Scheduling

Scheduler duy trì 140 MLQ ready queues. Hàm `get_mlq_proc()` duyệt từ priority 0 đến 139, lấy FIFO process đầu queue và giảm slot của mức đó. `running_list` là danh sách, vì vậy process được xóa bằng `purgequeue()` trước khi enqueue trở lại.

3.1 Gantt diagram minh họa

Với test `sched_0`, time slice bằng 2 và một CPU, phần đầu lịch chạy có dạng:

Slot	1-2	3-4	5-6	7-8	9-10	11-12	13-14	15-16	17-18
Process	P1	P2	P3	P4	P3	P4	P3	P4	P3
Priority	4	4	0	0	0	0	0	0	0

Khi P3 và P4 priority 0 xuất hiện, chúng được chọn trước P1 và P2 priority 4. Các process cùng priority luân phiên FIFO.

4 System Call và phân tách User/Kernel

Các memory library wrapper ở user side chỉ dùng PCB để lấy PID và kernel handle, sau đó truyền scalar arguments qua syscall 17. Kernel gọi `find_proc_by_pid()` để duyệt `running_list`, `ready_queue` và MLQ queues dưới scheduler lock. PCB không được truyền như một syscall argument.

Raw physical read/write chỉ thành công nếu frame vật lý hiện đang được map trong page table của PID gọi syscall. Test `os_mem_protection` cho kết quả:

```
Kernel denied PID 1 physical read at 0x0
Kernel denied PID 1 physical write at 0x0
```

Kernel allocation sử dụng địa chỉ thuộc vùng `PAGING_KMEM_BASE` và yêu cầu một dãy physical frame liên tiếp. Kernel region có physical mapping riêng và không xuất hiện trong user page table.

5 Memory Management

Mỗi process có VMA, symbol region table và free-region list riêng. Khi `alloc` không tìm được free region đủ lớn, kernel tăng VMA limit, lấy frame từ RAM và map page. `free` đưa region trở lại free-region list để tái sử dụng. Read/write kiểm tra region ID và offset trước khi dịch địa chỉ.

6 Paging 64-bit năm cấp

6.1 Address translation

Địa chỉ canonical 57-bit được chia thành:

Phần	Bit	Số entry
PGD	56–48	512
P4D	47–39	512
PUD	38–30	512
PMD	29–21	512
PT	20–12	512
Offset	11–0	4096 byte/page theo sơ đồ địa chỉ

Implementation thực hiện walk thật theo chuỗi PGD → P4D → PUD → PMD → PT. Không gian mô phỏng hiện tại chỉ cần một nhánh sparse ở các cấp trên. Địa chỉ có các bit 63–57 không phải sign extension của bit 56 bị từ chối. Test `os_mm64_canonical`:

Sơ đồ địa chỉ 64-bit dùng offset 12 bit theo mô hình 4 KiB/page. Thiết bị physical-memory nhỏ trong simulator vẫn dùng frame granularity của framework gốc để các test RAM 1 KiB và 4 KiB tiếp tục kiểm tra được tình huống thiếu frame.

```
MM64 rejected non-canonical address 0x1000000000000000
```

6.2 Thống kê

Mỗi process dùng năm bảng, mỗi bảng 512 entry, mỗi entry 8 byte:

$$5 \times 512 \times 8 = 20480 \text{ byte.}$$

Mỗi page-table walk hợp lệ truy cập năm cấp, nên `accesses = 5 * walks`. Ví dụ output:

```
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0]
walks=7 accesses=35 bytes=20480
```

7 Kiểm thử

Nhóm test	Kết quả	Mục tiêu
<code>sched, sched_0, sched_1</code>	PASS	MLQ và multi-CPU
<code>os_0/1/2_mlq_paging</code>	PASS	Scheduler kết hợp paging
<code>os_syscall, os_syscall_list, os_sc</code>	PASS	Syscall dispatch
<code>os_mm64_canonical</code>	PASS	Canonical-address validation
<code>os_mem_protection</code>	PASS	Từ chối raw physical access

Toàn bộ test được chạy bằng:

```
make clean
make all
make test
```

8 Kết luận

Hệ thống hoàn thành MLQ scheduling, unified syscall bằng PID, bảo vệ user/kernel space, cấp phát user/kernel memory, paging 64-bit năm cấp, canonical validation và thống kê page-table. Output đa CPU có thể khác thứ tự giữa các lần chạy nhưng mọi kết quả phải tuân theo priority, time slice và các điều kiện bảo vệ đã mô tả.